

Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

DEPARTMENT OF ELECTRONICS AND

COMMUNICATION ENGINEERING

**PROJECT REPORT
ON**

**RFID ATTENDANCE SYSTEM USING ESP32 AND
GOOGLE SHEETS**

SUBMITTED BY:

TAMIZHARASAN T V -113123UG04110

SUNDAR P-113123UG04108

RAMESH KUMAR M –113123UG04086

UNDER THE GUIDANCE OF:

GUIDE NAME: MR.D.RUBAN THOMAS

INDEX

S.NO	TABLE OF CONTENT	PAGES
1.	Introduction	3
2.	Problem Statement	4
3.	Objectives	5
4.	RFID-Based Attendance Systems	5
5.	Components Description	6
6.	Circuit Diagram And Hardware Connection	10
7.	Working Principle	11
8.	Software Requirements	12
9.	Program Development	14
10.	Google Sheets Integration	20
11.	Results And Output	22
12.	Advantages	22
13.	Applications	23

CHAPTER 1: INTRODUCTION

1.1 Overview of Embedded Systems

An embedded system is a dedicated computing system designed to perform specific tasks within a larger mechanical or electrical system. Unlike general-purpose computers, embedded systems are optimized for a narrow range of functions and are tightly integrated with the hardware they control. They are characterized by real-time computing constraints, low power consumption, compact form factors, and high reliability.

Modern embedded systems form the backbone of IoT devices, automotive control units, medical instruments, industrial automation controllers, and consumer electronics. The rapid advancement in microcontroller technology, particularly the availability of low-cost, high-performance SoCs such as the ESP8266 and ESP32, has democratized embedded system development and accelerated IoT adoption globally.

1.2 Internet of Things (IoT)

The Internet of Things (IoT) refers to the network of physical devices, vehicles, appliances, and other objects embedded with sensors, software, and network connectivity that enables them to collect, exchange, and act upon data. IoT bridges the gap between the physical and digital worlds by enabling remote monitoring, control, and automation over standard internet protocols.

Key pillars of an IoT architecture include: (i) Sensing Layer – physical sensors and actuators, (ii) Network Layer – Wi-Fi, Bluetooth, Zigbee, LoRa, (iii) Processing Layer – edge and cloud computing, and (iv) Application Layer – user interfaces and dashboards. The global IoT market is projected to surpass 25 billion connected devices by 2030, with applications spanning smart cities, precision agriculture, healthcare, and intelligent transportation systems.

1.3 Attendance Management Systems

Attendance management is a critical administrative function in educational institutions, corporate organizations, and government bodies. Accurate attendance records are essential for regulatory compliance, performance evaluation, payroll processing, and academic assessment. Conventional methods rely on paper registers, which are susceptible to errors, manipulation, and data loss.

The evolution of attendance systems can be traced from manual paper registers to computerized spreadsheets, biometric systems, and now IoT-enabled cloud-based platforms. RFID-based attendance systems represent a significant leap in automation, providing contactless, near-instantaneous identification with minimal infrastructure requirements.

1.4 Need for Automation

The need for automated attendance is underscored by multiple factors: institutions managing hundreds of students per class cannot afford the time overhead of manual roll-calls; proxy attendance remains a persistent problem; manual records are inaccessible in real time to parents, administrators, or compliance officers; and physical records are vulnerable to fire, water damage, and wilful manipulation. An IoT-based RFID attendance system eliminates all of these issues while providing instantaneous, geographically distributed access to attendance data via cloud storage.

CHAPTER 2: PROBLEM STATEMENT

Existing manual and semi-automated attendance systems in educational institutions suffer from the following critical deficiencies:

- Time inefficiency: Manual roll-calls consume 5–10 minutes of every class period, reducing effective teaching time significantly.
- Proxy attendance: Students frequently mark attendance on behalf of absent peers, leading to inflated attendance records and academic dishonesty.
- Data integrity: Paper records are prone to tampering, accidental damage, and transcription errors during digitization.
- Limited accessibility: Attendance data is confined to physical registers or isolated spreadsheets with no real-time access for parents, administrators, or auditors.
- No automated alerts: There is no mechanism to immediately flag chronic absenteeism for intervention.
- Administrative burden: Compiling monthly attendance summaries requires manual compilation, consuming significant faculty time.

These limitations demand a robust, automated, cloud-integrated attendance solution that is cost-effective, tamper-proof, and accessible in real time.

CHAPTER 3: OBJECTIVES

The primary objectives of this project are as follows:

- To design and implement a contactless, RFID-based attendance system using the ESP8266 NodeMCU microcontroller.
- To automatically identify students via unique RFID tag UUIDs and log their attendance without manual intervention.
- To integrate the system with Google Sheets via Google Apps Script for real-time, cloud-based data storage and retrieval.
- To provide instant visual feedback through a 16×2 LCD display and audio feedback through a buzzer upon successful attendance marking.
- To enable authorized personnel to access, monitor, and export attendance records from any internet-connected device.
- To develop a scalable, low-cost solution suitable for deployment across classrooms, laboratories, and administrative departments.
- To demonstrate the practical integration of IoT protocols (HTTPS/REST) with embedded microcontroller systems.

CHAPTER 4: RFID-Based Attendance Systems

RFID-based attendance systems use Radio Frequency Identification technology to automatically identify and record the attendance of users through contactless communication. These systems are widely used in educational institutions, offices, industries, and laboratories because of their speed, accuracy, and automation capabilities.

The RFID system used in this project operates at **13.56 MHz**, which belongs to the **High Frequency (HF) band**, and follows the **ISO/IEC 14443A standard**. This standard supports secure and reliable short-range wireless communication between the RFID reader and RFID tags/cards.

In the proposed system, the MFRC522 RFID reader reads the unique identification number (UID) stored inside the RFID tag when the card is placed near the reader. The reading range is typically up to **10 cm**, and the card detection process takes less than **100 milliseconds**, enabling fast attendance recording.

Previous research works by Bhatt et al. (2021) and Rajan et al. (2022) implemented RFID attendance systems using local database storage methods. Although these systems successfully automated attendance management, the data was limited to local storage and could not be monitored remotely in real time.

The present work improves upon these existing systems by integrating the RFID attendance system with cloud-based storage using **Google Sheets**. The attendance data captured by the ESP8266 is transmitted through WiFi using Google Apps Script and stored directly in Google Sheets. This integration enables:

- Real-time attendance monitoring
- Remote data access from any location
- Cloud-based attendance storage
- Easy report generation and analysis
- Reduced dependency on local databases

The system also includes an LCD display to provide immediate feedback to users by displaying messages such as student name, attendance status, and system notifications. Compared to traditional manual attendance methods, the RFID-based system significantly reduces human effort, minimizes errors, and increases operational efficiency.

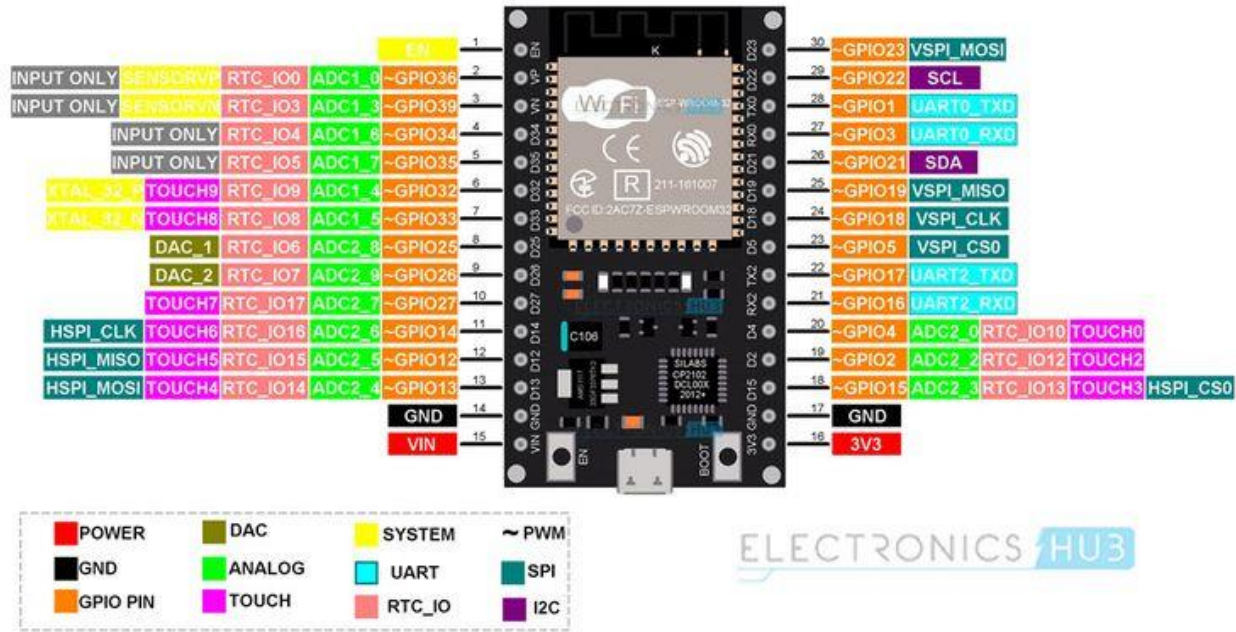
Due to its low cost, simple implementation, and IoT capabilities, the RFID attendance system is highly suitable for smart campus applications and modern automated monitoring systems.

CHAPTER 5: COMPONENTS DESCRIPTION

5.1 ESP32 BOARD

ESP32 is a low-cost and high-performance microcontroller mainly used for Internet of Things (IoT) and embedded system applications. It is developed to provide wireless communication features such as WiFi and Bluetooth within a single chip. Due to its fast processing speed, low power consumption, and multiple communication interfaces, ESP32 is widely used in automation, monitoring, and smart electronic systems.

In this project, ESP32 acts as the main controller unit of the RFID attendance system. It receives data from the RFID reader, processes the student information, and uploads attendance data to Google Sheets through WiFi connectivity.



5.2 RC522 RFID Reader Module

The MFRC522 is a highly integrated reader/writer IC for contactless communication at 13.56 MHz, designed by NXP Semiconductors. It operates on the ISO/IEC 14443A standard and supports MIFARE Classic, MIFARE Ultralight, and MIFARE DESFire card families. Communication with the host microcontroller is achieved via SPI (up to 10 Mbit/s), I2C, or UART interfaces.

Parameter	Specification
Operating Frequency	13.56 MHz
Standard	ISO/IEC 14443 Type A
Communication	SPI (10 Mbit/s), I2C (400 kHz), UART (1228.8 kD)
Operating Voltage	2.5V – 3.6V
Operating Current	13–26 mA (read), <80 μ A (power-down)
Read Range	Up to 50 mm (card-dependent)
Data Rate	106 kD, 212 kD, 424 kD, 848 kD
Operating Temp.	-20°C to +85°C

interfaced using a PCF8574 I2C expander module. The I2C interface minimizes the number of GPIO pins required for communication with the microcontroller.

In normal parallel communication mode, the LCD requires approximately 7 GPIO pins for operation. By using the I2C module, the communication is simplified to only two pins: SDA (Serial Data) and SCL (Serial Clock). This helps conserve the limited GPIO pins available in the ESP8266 microcontroller.



5.4 Active Buzzer

An active buzzer contains an internal oscillator and produces a fixed-frequency tone when a DC voltage is applied. In this system, the buzzer provides auditory confirmation on each successful RFID scan. It operates at 5V DC and draws approximately 30 mA. A BC547 NPN transistor is used as a switch to drive the buzzer from the 3.3V ESP8266 GPIO output, protecting the microcontroller from excessive current draw.

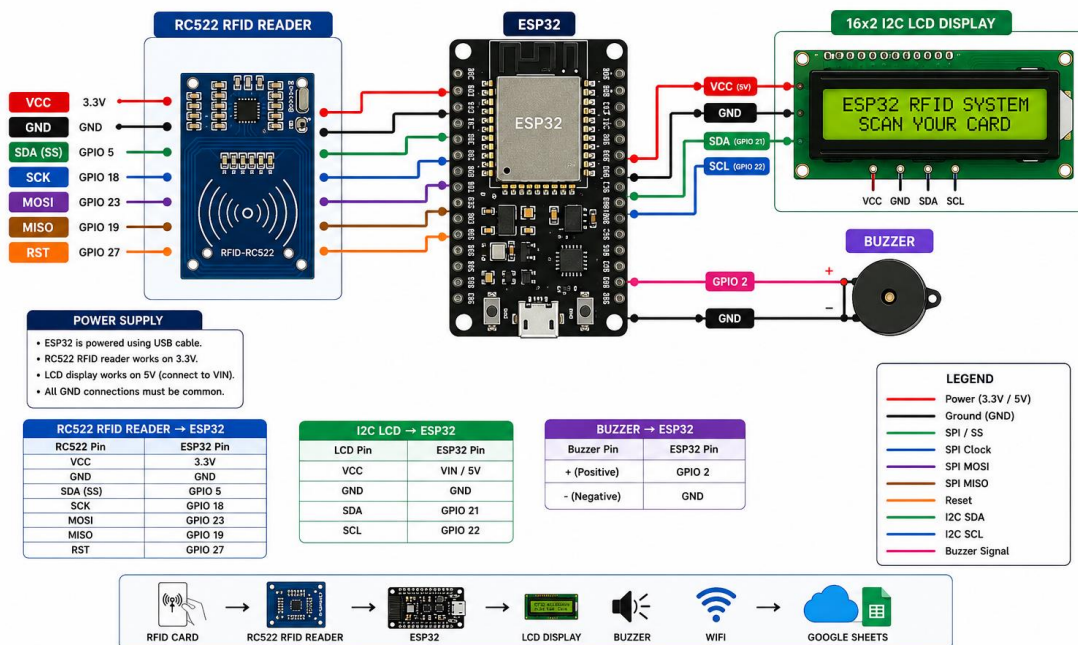


5.5 Power Supply

The ESP8266 NodeMCU is powered via the onboard micro-USB connector (5V, 1A minimum). The 3.3V rail is derived by the onboard AMS1117-3.3 LDO regulator and supplies the RC522 module. The LCD and buzzer are powered from the 5V VIN pin. Total system current consumption is approximately 250 mA under normal operation.

CHAPTER 6: CIRCUIT DIAGRAM AND HARDWARE CONNECTIONS

ESP32 RFID ATTENDANCE SYSTEM — CIRCUIT DIAGRAM —



CHAPTER 7: WORKING PRINCIPLE

The system operates in the following sequential steps:

Step 1 – Power ON and Initialization

The ESP32 microcontroller boots and executes the `setup()` function during system initialization. The SPI communication bus is initialized to establish communication between the ESP32 and the RC522 RFID reader module. The SPI pins used are GPIO 18 for SCK, GPIO 19 for MISO, GPIO 23 for MOSI, and GPIO 5 for SS (Slave Select). The RC522 RFID module is initialized using the MFRC522 library, which enables the ESP32 to detect and read RFID cards and tags. After successful initialization, the RFID reader waits for a card scanning operation.

The I2C communication bus is initialized using GPIO 21 as SDA (Serial Data) and GPIO 22 as SCL (Serial Clock). The LiquidCrystal_I2C library is used to initialize the 16×2 LCD display with the default I2C address 0x27. After initialization, the LCD backlight is enabled and a welcome message is displayed on the screen:

RFID Attendance
Scan Your Card

This message indicates that the RFID attendance system is ready for operation and waiting for the user to scan an RFID card.

Step 2 – Wi-Fi Connection

The ESP8266 calls `WiFi.begin(ssid, password)` and enters a polling loop awaiting `WL_CONNECTED` status. A 500 ms delay prevents busy-waiting. Upon successful connection, the assigned IP address is printed to the Serial Monitor and a confirmation message appears on the LCD for 2 seconds.

Step 3 – RFID Scanning

In the main `loop()`, the MFRC522 library continuously polls for new cards using `rfid.PICC_IsNewCardPresent()`. When a card enters the reader field (within ~3–5 cm), the reader activates the card via ISO 14443A anti-collision protocol.

Step 4 – UID Reading

`rfid.PICC_ReadCardSerial()` reads the 4-byte UID from the card. The UID bytes are concatenated into a hexadecimal string (e.g., 'A3B2C1D0') for comparison.

Step 5 – Student Identification

The UID string is matched against a predefined lookup array in program memory. Upon a successful match, the corresponding student name and roll number are retrieved. If no match is found, the LCD displays 'Unknown Card!' and the buzzer emits three short beeps.

Step 6 – LCD and Buzzer Feedback

The matched student name is displayed on the first LCD row and roll number on the second row. The buzzer emits one short beep (100 ms) confirming successful attendance marking. The message remains on screen for 3 seconds before reverting to standby.

Step 7 – Data Transmission to Google Sheets

A `WiFiClientSecure` object establishes an HTTPS connection to `script.google.com`. The URL includes GET parameters: `studentName`, `rollNumber`, `date`, and `time` (obtained from NTP server). The Apps Script web app receives the request, parses the parameters, and appends a new row to the attendance spreadsheet.

Step 8 – Next Scan Ready

The RFID card is halted (`rfid.PICC_HaltA()`) and encryption is stopped (`rfid.PCD_StopCrypto1()`). The system returns to Step 3, ready for the next scan.

CHAPTER 8: SOFTWARE REQUIREMENTS

8.1 Arduino IDE

Arduino IDE (version 2.x recommended) is an open-source integrated development environment supporting C/C++ programming for microcontrollers. It provides a sketch editor, compiler (GCC-based toolchain), serial monitor/plotter, and board manager for adding third-party hardware packages. Download: <https://www.arduino.cc/en/software>

8.2 ESP8266 Board Package

Add the following URL to Arduino IDE → File → Preferences → Additional Boards Manager URLs:

https://arduino.esp8266.com/stable/package_esp8266com_index.json

Then install 'esp8266 by ESP8266 Community' via Tools → Board Manager. Select 'NodeMCU 1.0 (ESP-12E Module)' as the target board.

8.3 Required Arduino Libraries

Library	Version	Purpose	Installation
MFRC522	1.4.10+	RC522 RFID reader control	Library Manager
LiquidCrystal_I2C	1.1.2+	16×2 I2C LCD control	Library Manager
ESP8266WiFi	Built-in	Wi-Fi connectivity	With ESP8266 package
ESP8266HTTPSClient	Built-in	HTTPS client	With ESP8266 package
WiFiClientSecure	Built-in	TLS/SSL connection	With ESP8266 package
NTPClient	3.2.1+	Network time sync	Library Manager
WiFiUdp	Built-in	UDP for NTP	With ESP8266 package

8.4 Google Apps Script

Google Apps Script is a JavaScript-based cloud scripting platform that allows automation of Google Workspace applications including Sheets, Docs, and Gmail. The script is deployed as a Web App with 'Anyone, even anonymous' access to receive HTTP GET/POST requests from the ESP8266. No server infrastructure is required beyond a Google account.

8.5 Google Sheets

Google Sheets serves as the cloud database. A spreadsheet is created with columns: S.No., Student Name, Roll Number, Date, Time, Status. The Apps Script appends rows programmatically via the SpreadsheetApp API. The sheet is accessible from any browser or the Google Sheets mobile app with appropriate sharing permissions.

CHAPTER 9: PROGRAM DEVELOPMENT

9.1 Complete ESP8266 Source Code

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <map>
#include "time.h"

// ===== WiFi =====

const char* ssid = "Neo!";
const char* password = "neo111111";

String serverName =
"https://script.google.com/macros/s/AKfycbyKrZ03geZt0oNEGctifnFSBvu8FRBAoQh-
EqW_vq8cEtM/exec";

// ===== RFID =====

#define SS_PIN 5
#define RST_PIN 27

MFRC522 rfid(SS_PIN, RST_PIN);

// ===== LCD =====

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ===== Buzzer =====

#define BUZZER_PIN 2

// ===== Time =====

const char* ntpServer = "pool.ntp.org";
const long gmtoffset_sec = 19800;
```

```

const int daylightOffset_sec = 0;

// ===== UID Mapping =====

std::map<String, String> uidToName = {

  {"40EDDF61", "tamizharasan"},

  {"0B263806", "sundar"}

};

// ===== Entry / Exit =====

std::map<String, bool> entryState;

// ===== Setup =====

void setup() {
  Serial.begin(115200);

  // LCD

  Wire.begin(21, 22);

  lcd.init();

  lcd.backlight();

  lcd.setCursor(0, 0);

  lcd.print("Connecting");

  lcd.setCursor(0, 1);

  lcd.print("WiFi...");

  // WiFi

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

  }

  lcd.clear();

  lcd.setCursor(0, 0);

```

```

lcd.print("WiFi Connected");
delay(2000);
// Time Setup
configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
Serial.print("Waiting for NTP Time");
struct tm timeinfo;
while (!getLocalTime(&timeinfo)) {
Serial.print(".");
delay(500);
}
Serial.println();
Serial.println(asctime(&timeinfo));
// RFID Setup
SPI.begin(18, 19, 23, 5);
rfid.PCD_Init();
// Buzzer
pinMode(BUZZER_PIN, OUTPUT);
// Initial LCD
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("RFID Attendance");
lcd.setCursor(0, 1);
lcd.print("Scan Card");
}
// ===== Get Time =====
String getTime() {
struct tm timeinfo;

```

```

if (!getLocalTime(&timeinfo)) {
return "00:00:00";
}

char buffer[9];
strftime(buffer, sizeof(buffer), "%H:%M:%S", &timeinfo);
return String(buffer);
}

// ===== Get Date =====

String getDate() {
struct tm timeinfo;
if (!getLocalTime(&timeinfo)) {
return "00/00/00";
}

char buffer[9];
strftime(buffer, sizeof(buffer), "%d/%m/%y", &timeinfo);
return String(buffer);
}

// ===== Main Loop =====

void loop() {
// Default LCD
lcd.setCursor(0, 0);
lcd.print("Scan Card   ");
lcd.setCursor(0, 1);
lcd.print("           ");
// Check RFID
if (!rfid.PICC_IsNewCardPresent()) {
return;

```

```

}
if (!rfid.PICC_ReadCardSerial()) {
return;
}
// Read UID
String uid = "";
for (byte i = 0; i < rfid.uid.size; i++) {
char buf[3];
sprintf(buf, "%02X", rfid.uid.uidByte[i]);
uid += buf;
}
Serial.println("Card Detected!");
Serial.println("UID: " + uid);
// Get Name
String name = "Unknown";
if (uidToName.count(uid)) {
name = uidToName[uid];
}
// Entry / Exit
bool isEntry = !entryState[uid];
entryState[uid] = isEntry;
String status = isEntry ? "ENTRY" : "EXIT";
// Date & Time
String date = getDate();
String time = getTime();
// LCD Display
lcd.clear();

```

```

lcd.setCursor(0, 0);
lcd.print(status + " " + name);
lcd.setCursor(0, 1);
lcd.print(time);

// Buzzer
digitalWrite(BUZZER_PIN, HIGH);
delay(200);
digitalWrite(BUZZER_PIN, LOW);

// Send to Google Sheet
if (WiFi.status() == WL_CONNECTED) {
  HTTPClient http;
  String url =
  serverName +
  "?name=" + name +
  "&entry=" + status +
  "&rfid=" + uid +
  "&date=" + date +
  "&time=" + time;
  http.begin(url);
  int httpCode = http.GET();
  Serial.print("HTTP Response Code: ");
  Serial.println(httpCode);
  http.end();
}

// Delay
delay(2000);

// Reset LCD

```

```
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("RFID Attendance");  
lcd.setCursor(0, 1);  
lcd.print("Scan Card");  
}
```

CHAPTER 10: GOOGLE SHEETS INTEGRATION

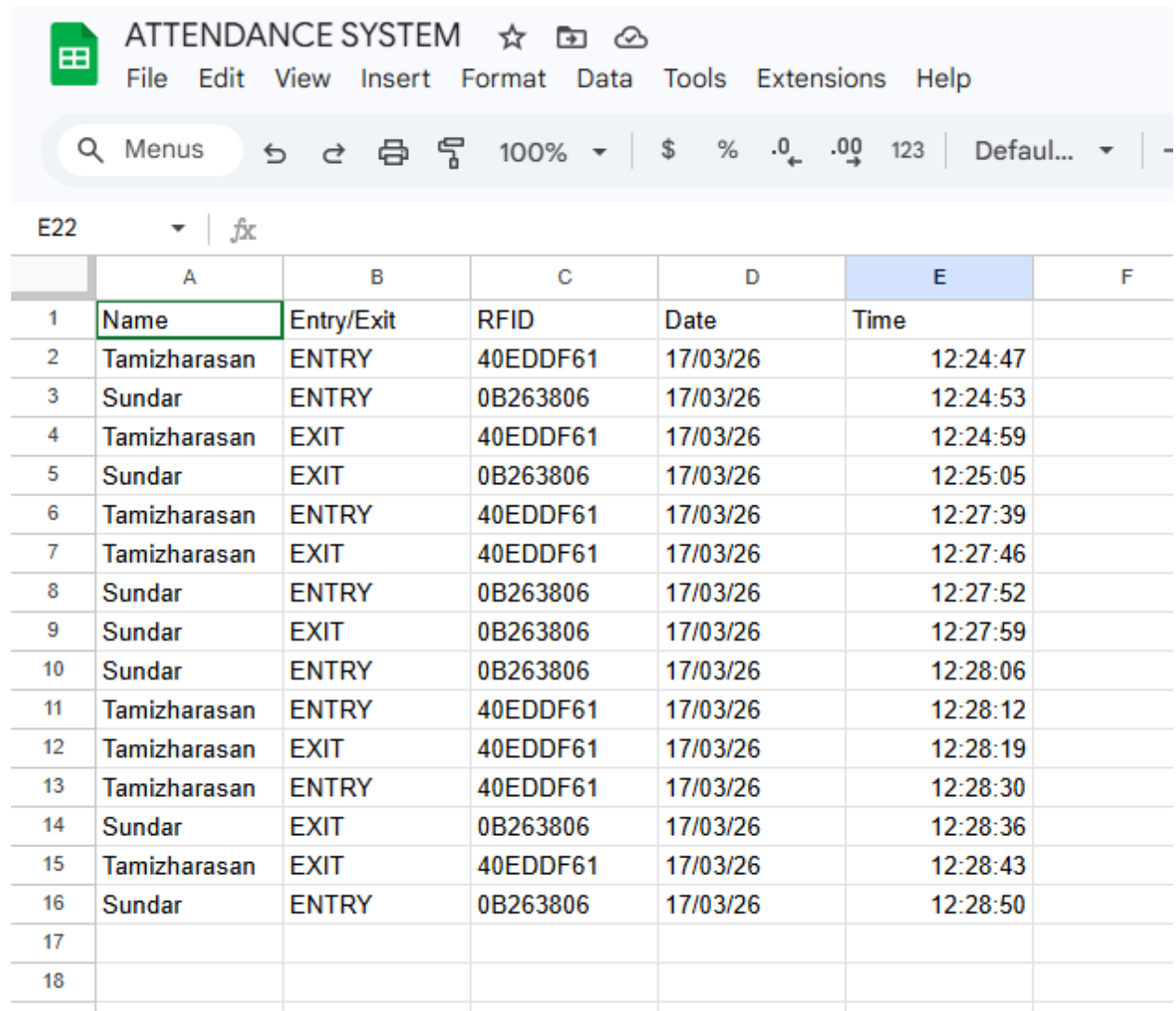
10.1 Google Apps Script Code

```
function doGet(e) {  
  
  var sheet = SpreadsheetApp  
    .openById('YOUR_SPREADSHEET_ID')  
    .getActiveSheet();  
  
  var lastRow = sheet.getLastRow();  
  
  var name = e.parameter.name;  
  var roll = e.parameter.roll;  
  var date = e.parameter.date;  
  var time = e.parameter.time;  
  
  sheet.getRange(lastRow+1, 1).setValue(lastRow);  
  
  sheet.getRange(lastRow+1, 2).setValue(name);  
  
  sheet.getRange(lastRow+1, 3).setValue(roll);  
  
  sheet.getRange(lastRow+1, 4).setValue(date);  
  
  sheet.getRange(lastRow+1, 5).setValue(time);  
  
  sheet.getRange(lastRow+1, 6).setValue('Present');  
  
  return ContentService  
    .createTextOutput('Success');  
}
```

10.2 Deployment Process

1. Open the target Google Sheets spreadsheet and create the attendance table. Add the column headers such as Name, Entry, RFID, Date, and Time in Row 1.
2. Navigate to Extensions → Apps Script in Google Sheets.
3. Remove the default sample code and paste the required `doGet()` function for receiving attendance data from the ESP32.
4. Save the Apps Script project with a suitable name such as “RFID Attendance System”.
5. Click Deploy → New Deployment.
6. Select the deployment type as Web App.
7. Configure the deployment settings:
 - a. Execute as: Me
 - b. Who has access: Anyone
8. Click Deploy to create the Web App deployment.
9. Authorize the required permissions when prompted by Google for spreadsheet access.
10. After successful deployment, copy the generated Web App URL.
11. Extract the Script ID from the URL. The Script ID is the long string present between `/s/` and `/exec`.
12. Paste the Script ID or complete Web App URL into the ESP32 firmware in the `serverName` or `GAS_ID` variable.
13. Upload the program to the ESP32 using Arduino IDE.
14. Power ON the system and connect the ESP32 to WiFi.
15. Scan the RFID card using the RC522 RFID reader.
16. The attendance data is automatically transmitted to Google Sheets and stored in the spreadsheet.
17. The RFID attendance system is now ready for operation.

CHAPTER 11: RESULTS AND OUTPUT



	A	B	C	D	E	F
1	Name	Entry/Exit	RFID	Date	Time	
2	Tamizharasan	ENTRY	40EDDF61	17/03/26	12:24:47	
3	Sundar	ENTRY	0B263806	17/03/26	12:24:53	
4	Tamizharasan	EXIT	40EDDF61	17/03/26	12:24:59	
5	Sundar	EXIT	0B263806	17/03/26	12:25:05	
6	Tamizharasan	ENTRY	40EDDF61	17/03/26	12:27:39	
7	Tamizharasan	EXIT	40EDDF61	17/03/26	12:27:46	
8	Sundar	ENTRY	0B263806	17/03/26	12:27:52	
9	Sundar	EXIT	0B263806	17/03/26	12:27:59	
10	Sundar	ENTRY	0B263806	17/03/26	12:28:06	
11	Tamizharasan	ENTRY	40EDDF61	17/03/26	12:28:12	
12	Tamizharasan	EXIT	40EDDF61	17/03/26	12:28:19	
13	Tamizharasan	ENTRY	40EDDF61	17/03/26	12:28:30	
14	Sundar	EXIT	0B263806	17/03/26	12:28:36	
15	Tamizharasan	EXIT	40EDDF61	17/03/26	12:28:43	
16	Sundar	ENTRY	0B263806	17/03/26	12:28:50	
17						
18						

CHAPTER 12: ADVANTAGES

- Speed: Each attendance marking takes under 2 seconds, compared to 5–10 minutes for manual roll calls in a 60-student class.
- Elimination of Proxy Attendance: Physical card proximity (≤ 5 cm) is required, making proxy attendance significantly more difficult.
- Real-time Cloud Storage: Attendance data is immediately available on Google Sheets from any browser worldwide.
- Low Cost: Total hardware cost is approximately ■600–■800, compared to ■5,000–■50,000 for biometric systems.
- Ease of Implementation: The system uses widely available, open-source hardware and software, requiring no specialized expertise.

- Scalability: Additional students are enrolled by simply adding UID-Name-Roll entries to the firmware lookup table.
- No Physical Contact Required: Contactless operation is hygienic and suitable for post-pandemic institutional environments.
- Automated Record Keeping: No manual data entry is required; records are structured and ready for analysis immediately.
- Accessibility: Google Sheets can be shared with parents, administrators, and auditors with role-based access control.

CHAPTER 13: APPLICATIONS

- Educational Institutions – Automated attendance management for schools, colleges, and universities with real-time attendance monitoring and parent notification capability.
- Corporate Offices – Employee entry and exit tracking integrated with payroll and attendance management systems.
- Industrial Facilities – Workforce monitoring and access control in manufacturing industries, factories, and warehouses.
- Libraries – Automated entry and exit logging along with library resource access management for users.
- Healthcare Sector – Staff attendance tracking in hospitals and clinics with proper compliance and audit record maintenance.
- Examination Halls – Candidate attendance recording and identity verification during board and competitive examinations.
- Government Offices – Attendance monitoring of employees for improved transparency and accountability in administrative operations.
- Laboratories – Restricted laboratory access management for equipment safety, monitoring, and usage auditing.

